

Bachelor Thesis

im Studiengang Geoinformatik

zur Erlangung des Grades eines
Bachelor of Science (B.Sc.)

über das Thema

LLM-gestützte Generierung von Playwright-E2E-Tests aus Use Cases

Einfluss von UI-Kontext am Beispiel von WebGIS-Anwendungen

von

Nils Große Beikel

Betreuer (Universität):	Dr. Christian Knoth
Betreuer (Unternehmen):	Dr. Thore Fechner
Matrikelnummer :	546548
Abgabedatum :	31. August 2026

Inhaltsverzeichnis

Abbildungsverzeichnis	IV
Tabellenverzeichnis	V
Abkürzungsverzeichnis	VI
1 Einleitung	1
1.1 Motivation und Problemstellung	1
1.2 Zielsetzung und Erkenntnisinteresse	1
1.3 Aufbau der Arbeit	1
2 Grundlagen	2
2.1 Large Language Models	2
2.1.1 Funktionsprinzip und Kontext als Qualitätsfaktor	2
2.1.2 Einsatz im Software Engineering	3
2.2 E2E-Testautomatisierung mit Playwright	4
2.2.1 Teststufen und Einordnung von E2E-Tests	4
2.2.2 Playwright: Selektoren, Assertions, asynchrone UIs	5
2.3 Open Pioneer Trails	6
2.3.1 Framework-Überblick und Komponentenmodell	6
2.3.2 data-testid-Konventionen und Testbarkeit	6
2.4 WebGIS-Anwendungen und Testkomplexität	6
3 Stand der Forschung	7
3.1 LLM-gestützte Testgenerierung	7
3.2 Forschungslücke und Einordnung dieser Arbeit	7
4 Methodik und Forschungskonzept	8
4.1 Überblick und Forschungsdesign	8
4.2 Use Cases: Definition und JSON-Format	8
4.3 Versuchsaufbau: Die vier Kontextstufen	8
4.3.1 Stufe 1 - Baseline (kein Kontext)	8
4.3.2 Stufe 2 - Accessibility Snapshot via MCP	8
4.3.3 Stufe 3 - Automatisch generierte UI-Map	8
4.3.4 Stufe 4 - Manuell erstellte UI-Map	8
4.4 LLM-Konfiguration (Modell, Temperatur, Prompt-Struktur)	8
4.5 Bewertungskriterien	8

5 Implementierung der Evaluationsumgebung	9
5.1 Demo-WebGIS-Anwendung	9
5.2 Use-Case-Sammlung	9
5.3 Manuell erstellte Referenztests	9
6 Durchführung und Evaluation	10
6.1 LLM-gestützter Generierungsworkflow	10
6.2 Ergebnisse je Kontextstufe	10
6.3 Vergleich der Kontextstufen	10
6.4 Vergleich mit manuell erstellten Tests	10
6.5 GIS-spezifische Besonderheiten	10
7 Diskussion	11
7.1 Interpretation der Ergebnisse	11
7.2 Limitationen	11
7.3 Handlungsempfehlungen	11
8 Fazit und Ausblick	11
Anhang	12
Literaturverzeichnis	13

Abbildungsverzeichnis

Abbildung 1: Testpyramide nach Mike Cohn	5
--	---

Tabellenverzeichnis

Abkürzungsverzeichnis

CSS	Cascading Style Sheets
DOM	Document Object Model
E2E	End-to-End
LLM	Large Language Model
UI	User Interface

1 Einleitung

1.1 Motivation und Problemstellung

1.2 Zielsetzung und Erkenntnisinteresse

1.3 Aufbau der Arbeit

2 Grundlagen

2.1 Large Language Models

Large Language Models (LLMs) sind maschinelle Lernmodelle, die auf sehr großen Textmengen trainiert werden, um natürliche Sprache zu verarbeiten und zu erzeugen. Das Attribut „large“ bezieht sich dabei sowohl auf den Umfang der Trainingsdaten als auch auf die Anzahl der Modellparameter, die häufig im Bereich mehrerer Milliarden liegt. Technisch beruhen sie auf künstlichen neuronalen Netzen, genauer gesagt auf der Transformer-Architektur *Hou, X. et al., 2024*. Der folgende Abschnitt erläutert zunächst das Funktionsprinzip und die Rolle des bereitgestellten Kontexts, bevor der Einsatz im Software Engineering betrachtet wird.

2.1.1 Funktionsprinzip und Kontext als Qualitätsfaktor

Moderne LLMs basieren auf der Transformer-Architektur, die von *Vaswani, A. et al., 2017* eingeführt wurde. Ihr zentraler Bestandteil ist der Self-Attention-Mechanismus. Er erlaubt es dem Modell, die Beziehungen zwischen allen Elementen einer Eingabe unabhängig von ihrer Position zu gewichten. Anders als bei früheren rekurrenten Ansätzen wird die Eingabe dadurch nicht sequentiell, sondern weitgehend parallel verarbeitet, was das Training auf sehr großen Textmengen erst praktikabel gemacht hat *ebd.* Vor der Verarbeitung wird der Text in sogenannte Tokens zerlegt. Tokens sind in der Regel keine ganzen Wörter, sondern Teilwörter oder einzelne Zeichen, auf denen das Modell dann ausschließlich arbeitet.

Die Texterzeugung erfolgt autoregressiv. Auf Basis der bereits vorliegenden Tokens berechnet das Modell für das nächste Token eine Wahrscheinlichkeitsverteilung über das gesamte Vokabular. Aus dieser Verteilung wird ein Token ausgewählt und an die Eingabe angehängt. Der Vorgang wiederholt sich, bis die Ausgabe abgeschlossen ist. Wie stark die Auswahl vom jeweils wahrscheinlichsten Token abweichen darf, steuert unter anderem der Temperatur-Parameter. Ein niedriger Wert führt zu deterministischeren, ein hoher zu variableren Ausgaben. Das ist für diese Arbeit relevant, denn nur eine über alle Versuche konstant gehaltene Konfiguration erlaubt einen aussagekräftigen Vergleich der Generierungsergebnisse *ebd.*

Alle Tokens, die das Modell für eine Generierung berücksichtigt, befinden sich im sogenannten Kontextfenster. Dieses umfasst die Eingabe (den Prompt) und die bereits erzeugte Ausgabe und ist in seiner Größe begrenzt. Wissen, das nicht in den

Modellgewichten verankert ist, also nicht während des Trainings gelernt wurde, muss dem Modell zur Laufzeit über dieses Fenster bereitgestellt werden *Brown, T. B. et al., 2020*.

Eng damit verbunden ist die Fähigkeit zum In-Context Learning. ebd. zeigten, dass große Sprachmodelle Aufgaben allein anhand von Anweisungen oder Beispielen im Prompt lösen können, ohne dass die Modellgewichte angepasst werden. Das Modell „lernt“ hierbei nicht im klassischen Sinne, sondern nutzt die im Kontext bereitgestellten Informationen direkt als Grundlage für seine Vorhersage. Welche Inhalte im Prompt stehen, bestimmt damit maßgeblich die Ausgabe.

Daraus folgt ein zentraler Punkt für diese Arbeit. Ein LLM hat kein konkretes Wissen über eine individuell entwickelte Benutzeroberfläche. Informationen über die tatsächlich vorhandenen UI-Elemente, deren Selektoren und Zustände stehen weder in den Trainingsdaten noch sind sie zur Laufzeit zugänglich, solange sie nicht explizit im Prompt bereitgestellt werden. Die Qualität des generierten Testcodes hängt damit unmittelbar von Qualität und Umfang des bereitgestellten Kontexts ab.

2.1.2 Einsatz im Software Engineering

LLMs werden zunehmend im Software Engineering eingesetzt, insbesondere zur Generierung von Quellcode. Sichtbar wird das an Werkzeugen wie GitHub Copilot, das auf Grundlage des umgebenden Quellcode-Kontexts Vorschläge zur Codevervollständigung erzeugt und direkt in die Entwicklungsumgebung integriert ist. Der systematische Literaturüberblick von *Hou, X. et al., 2024* zeigt, dass der Einsatz von LLMs im Software Engineering inzwischen breit erforscht wird und deutlich über die reine Codevervollständigung hinausreicht.

Ein Anwendungsfeld ist die automatisierte Testgenerierung. LLMs werden hier eingesetzt, um aus natürlichsprachlichen oder strukturierten Beschreibungen Unit-Tests, Akzeptanztests und End-to-End (E2E)-Tests zu erzeugen. Bestehende Ansätze beziehen sich überwiegend auf generische Webanwendungen *Celik, A., Mahmoud, Q. H., 2025; Ferreira, M. et al., 2025*. GIS-spezifische Anwendungen werden aktuell kaum berücksichtigt. Eine ausführliche Betrachtung des Forschungsstands folgt in Kapitel 3.

Diesem Potential stehen bekannte Grenzen gegenüber. Da LLMs ihre Ausgaben auf Basis von Wahrscheinlichkeiten erzeugen, können sie plausibel wirkende, faktisch aber nicht existierende Inhalte produzieren, bei der Testgenerierung etwa erfundene Selektoren oder Aufrufe nicht vorhandener Funktionen. Dieses Phänomen wird als Halluzination bezeichnet *Hou, X. et al., 2024*. Hinzu kommt das fehlende Wissen über die konkrete Anwendung und

der Umstand, dass das Modell keinen Zugriff auf den tatsächlichen Laufzeitzustand der Oberfläche hat. Gerade für E2E-Tests, die gezielt auf konkrete User Interface (UI)-Elemente zugreifen müssen, sind diese Einschränkungen besonders relevant.

Grundsätzlich lassen sich Sprachmodelle auf zwei Wegen an eine spezifische Aufgabe anpassen: durch Fine-Tuning, also das Nachtrainieren der Modellgewichte auf aufgabenspezifischen Daten, oder durch Prompting, bei dem das Verhalten allein über die Gestaltung der Eingabe gesteuert wird. Prompting beruht auf dem in Abschnitt 2.1.1 beschriebenen In-Context Learning *Brown, T. B. et al., 2020*. Diese Arbeit verfolgt durchgängig den Prompting-Ansatz, da er gegenüber Fine-Tuning sowohl einen geringeren Ressourcenaufwand erfordert als auch eine bessere Generalisierbarkeit bietet. Ein prompt-basierter Workflow lässt sich ohne erneutes Training auf andere Anwendungen und Use Cases übertragen, während ein feinjustiertes Modell weitgehend auf seinen Trainingskontext beschränkt bliebe.

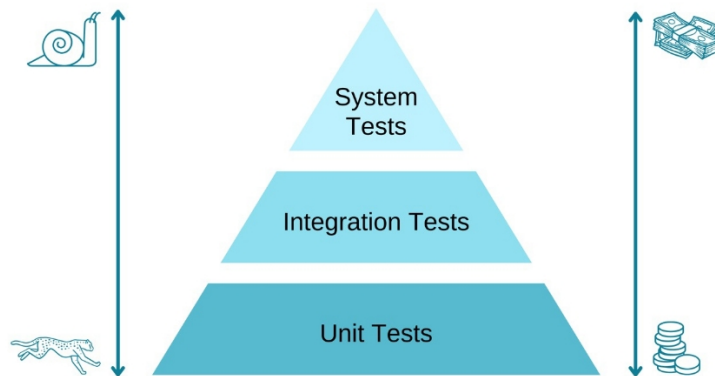
2.2 E2E-Testautomatisierung mit Playwright

Softwaretests lassen sich nach ihrem Umfang und ihrer Nähe zum Nutzerverhalten in verschiedene Stufen einteilen. Dieser Abschnitt ordnet E2E-Tests darin ein und stellt anschließend das Testframework Playwright vor, das in dieser Arbeit zur Generierung und Ausführung der Tests verwendet wird.

2.2.1 Teststufen und Einordnung von E2E-Tests

Eine verbreitete Einteilung von Softwaretests ist die Testpyramide, die von *Poenisch, M., 2022* beschrieben wird. Sie unterscheidet drei Ebenen, wie in Abbildung 1 dargestellt. Ganz unten stehen Unit-Tests, die einzelne Funktionen oder Komponenten prüfen. Sie laufen schnell und werden in großer Anzahl geschrieben. Darüber liegen die Integrationstests, die das Zusammenspiel mehrerer Komponenten betrachten. An der Spitze stehen E2E-Tests, die die Anwendung als Ganzes aus Sicht der Nutzer prüfen.

Abbildung 1: Die Testpyramide nach Mike Cohn (Quelle: *Poenisch, M., 2022*)



Ein E2E-Test steuert dazu einen Browser und führt dieselben Schritte aus wie ein Mensch. Dazu gehören auf Schaltflächen klicken, Felder ausfüllen und durch die Oberfläche navigieren. Geprüft wird, ob am Ende das erwartete Ergebnis eintritt. Dadurch geben E2E-Tests eine gute Auskunft darüber, ob eine fachliche Anforderung tatsächlich umgesetzt ist. Allerdings sind sie langsam, aufwendig zu erstellen und vergleichsweise anfällig und können schon bei kleinen Änderungen an der UI fehlschlagen. Deshalb wird empfohlen, wie nach der Form der Pyramide, die Anzahl gering zu halten.

Für WebGIS-Anwendungen sind E2E-Tests besonders relevant. Viele Probleme zeigen sich erst im Zusammenspiel der Komponenten und Interaktionen. Dazu gehört das asynchrone laden von geodaten, rendern der Karte oder die Auswirkungen der Layersteuerung auf die Kartenanzeige. Solche Abläufe lassen sich mit Unit-Tests kaum abbilden, weil sie den vollständigen, im Browser laufenden Zustand voraussetzen.

2.2.2 Playwright: Selektoren, Assertions, asynchrone UIs

Playwright ist ein Open Source E2E-Test-Framework von Microsoft zur Automatisierung von Browsern. Es steuert die Browser-Engines Chromium, Firefox und WebKit über eine einheitliche Programmierschnittstelle und führt Tests in einem echten Browser aus. Die Tests werden in dieser Arbeit in TypeScript geschrieben, das Playwright unterstützt *Microsoft, 2026*.

Um mit der Benutzeroberfläche zu interagieren, muss ein Test die jeweiligen Elemente finden. Playwright nutzt dafür sogenannte Lokatoren. Empfohlen wird, Elemente über nutzerseitige Eigenschaften anzusprechen, etwa über ihre ARIA-Rolle oder ihren

sichtbaren Text (`getByText`, `getByRole`). Solche Selektoren sind robuster gegenüber Änderungen am Markup als Cascading Style Sheets (CSS)- oder XPath-Ausdrücke, die an die konkrete Document Object Model (DOM)-Struktur gebunden sind. Eine Alternative sind Test-IDs. Über das Attribut `data-testid` lässt sich ein Element gezielt für Tests markieren und mit `getByTestId` ansprechen, unabhängig von Darstellung und Position. Welches Attribut als Test-ID gilt, ist konfigurierbar *Microsoft, 2026*.

Ein wesentliches Merkmal von Playwright ist das automatische Warten. Vor jeder Aktion prüft das Framework, ob das Zielelement sichtbar, stabil und bedienbar ist, ansonsten wird bis zu einem Timeout gewartet. Dadurch müssen nicht für jede Interaktion manuelle Wartezeiten eingebaut werden, was eine häufige Ursache für instabile Tests beseitigt. Dasselbe Prinzip gilt für Assertions. Mit `expect` formulierte Zusicherungen wie `expect(locator).toBeVisible()` oder `toHaveText()` werden so lange wiederholt, bis die Bedingung erfüllt ist oder die Zeit abläuft. Eine Assertion prüft damit nicht einen Momentzustand, sondern einen Zustand, der sich auch verzögert einstellen darf.

Für Fälle, die das automatische Warten nicht abdeckt, stellt Playwright extra Wartemethoden bereit, etwa auf eine bestimmte Netzwerkantwort (`waitForResponse`) oder einen Ladezustand der Seite (`waitForLoadState`). Das ist gerade bei WebGIS-Anwendungen wichtig, in denen Kartenkacheln und Geodaten asynchron über das Netz geladen werden und erst danach sichtbar auftauchen. Ein Test, der zu früh prüft, würde fehlschlagen, obwohl die Anwendung korrekt arbeitet. Diese Kombination aus automatischem Warten und gezielten Wartepunkten macht Playwright für den asynchronen Aufbau von WebGIS-Anwendungen gut geeignet.

2.3 Open Pioneer Trails

2.3.1 Framework-Überblick und Komponentenmodell

2.3.2 data-testid-Konventionen und Testbarkeit

2.4 WebGIS-Anwendungen und Testkomplexität

3 Stand der Forschung

3.1 LLM-gestützte Testgenerierung

3.2 Forschungslücke und Einordnung dieser Arbeit

4 Methodik und Forschungskonzept

4.1 Überblick und Forschungsdesign

4.2 Use Cases: Definition und JSON-Format

4.3 Versuchsaufbau: Die vier Kontextstufen

4.3.1 Stufe 1 - Baseline (kein Kontext)

4.3.2 Stufe 2 - Accessibility Snapshot via MCP

4.3.3 Stufe 3 - Automatisch generierte UI-Map

4.3.4 Stufe 4 - Manuell erstellte UI-Map

4.4 LLM-Konfiguration (Modell, Temperatur, Prompt-Struktur)

4.5 Bewertungskriterien

5 Implementierung der Evaluationsumgebung

5.1 Demo-WebGIS-Anwendung

5.2 Use-Case-Sammlung

5.3 Manuell erstellte Referenztests

6 Durchführung und Evaluation

6.1 LLM-gestützter Generierungsworkflow

6.2 Ergebnisse je Kontextstufe

6.3 Vergleich der Kontextstufen

6.4 Vergleich mit manuell erstellten Tests

6.5 GIS-spezifische Besonderheiten

7 Diskussion

7.1 Interpretation der Ergebnisse

7.2 Limitationen

7.3 Handlungsempfehlungen

8 Fazit und Ausblick

Anhang

Anhang 1: Beispielanhang

Dieser Abschnitt dient nur dazu zu demonstrieren, wie ein Anhang aufgebaut sein kann.

Anhang 1.1: Weitere Gliederungsebene

Auch eine zweite Gliederungsebene ist möglich.

Anhang 2: Bilder

Auch mit Bildern. Diese tauchen nicht im Abbildungsverzeichnis auf.

Abbildung 2: Beispielbild

Name	Änderungsdatum	Typ	Größe
 abbildungen	29.08.2013 01:25	Dateiordner	
 kapitel	29.08.2013 00:55	Dateiordner	
 literatur	31.08.2013 18:17	Dateiordner	
 skripte	01.09.2013 00:10	Dateiordner	
 compile.bat	31.08.2013 20:11	Windows-Batchda...	1 KB
 thesis_main.tex	01.09.2013 00:25	LaTeX Document	5 KB

Literaturverzeichnis

- Brown, Tom B., Mann, Benjamin, Ryder, Nick, Subbiah, Melanie, Kaplan, Jared, Dhariwal, Prafulla, Neelakantan, Arvind, Shyam, Pranav, Sastry, Girish, Askell, Amanda, Agarwal, Sandhini, Herbert-Voss, Ariel, Krueger, Gretchen, Henighan, Tom, Child, Rewon, Ramesh, Aditya, Ziegler, Daniel M., Wu, Jeffrey, Winter, Clemens, Hesse, Christopher, Chen, Mark, Sigler, Eric, Litwin, Mateusz, Gray, Scott, Chess, Benjamin, Clark, Jack, Berner, Christopher, McCandlish, Sam, Radford, Alec, Sutskever, Ilya, Amodei, Dario* (2020): Language Models are Few-Shot Learners, Version Number: 4, o. O., 2020, URL: <https://arxiv.org/abs/2005.14165>
- Celik, Arda, Mahmoud, Qusay H.* (2025): A Review of Large Language Models for Automated Test Case Generation, en, in: Machine Learning and Knowledge Extraction, 7 (2025), Nr. 3, S. 97
- Ferreira, Margarida, Viegas, Luís, Faria, João Pascoal, Lima, Bruno* (2025): Acceptance Test Generation with Large Language Models: An Industrial Case Study, in: 2025 IEEE/ACM International Conference on Automation of Software Test (AST), o. O.: IEEE, 2025-04, S. 1–11
- Hou, Xinyi, Zhao, Yanjie, Liu, Yue, Yang, Zhou, Wang, Kailong, Li, Li, Luo, Xiapu, Lo, David, Grundy, John, Wang, Haoyu* (2024): Large Language Models for Software Engineering: A Systematic Literature Review, in: ACM Trans. Softw. Eng. Methodol. 33 (2024), Nr. 8, 220:1–220:79
- Microsoft* (2026): Fast and reliable end-to-end testing for modern web apps | Playwright, en, Dokumentation, o. O., 2026-06, URL: <https://playwright.dev/>
- Poenisch, Marie* (2022): Die Testpyramide, de, o. O., 2022-08, URL: <https://www.openknowledge.de/blog/die-testpyramide>
- Vaswani, Ashish, Shazeer, Noam, Parmar, Niki, Uszkoreit, Jakob, Jones, Llion, Gomez, Aidan N., Kaiser, Lukasz, Polosukhin, Illia* (2017): Attention Is All You Need, Version Number: 7, o. O., 2017, URL: <https://arxiv.org/abs/1706.03762>